

ÍNDICE

1. FUNDAMENTOS DE JAVASCRIPT	2
1.1. Fundamentos del lenguaje.	2
1.2. Variables, tipos de datos y operadores.	3
1.3. Instrucciones condicionales.	6
1.4. Bucles.	7
1.5. Trabajando con arrays.	9
1.6. Funciones.	11
2. OBJETOS.....	13
2.1. Usando el espacio de nombres global.	13
2.2. Creación de objetos.	14
2.3. Creando objetos usando funciones constructoras.	14
2.4. Elementos HTML como objetos.	16
2.5. Objetos de utilidad.	18
2.6. Gestión de errores.	20

A large, light gray watermark of the Cloud Formación logo is positioned in the lower right area of the page, behind the index table.

1. Fundamentos de JavaScript

HTML y CSS proporcionan la estructura, semántica e información de presentación para una página web. Sin embargo, estas tecnologías no describen cómo el usuario interactúa con la página usando un navegador. Para implementar esta funcionalidad, todos los navegadores modernos incluyen un motor de JavaScript que soporta el uso de código script en las páginas. También implementan el Document Object Model (DOM), un estándar W3C que define como un navegador debe reflejar una página en memoria para permitir al motor de script acceder y alterar el contenido de la página.

1.1. Fundamentos del lenguaje.

1.1.1. ¿Qué es JavaScript?

JavaScript se originó como un lenguaje de programación a partir de 1990 y ha evolucionado constantemente. La norma en que se basa, ECMA-262, se aumenta y actualiza con frecuencia. Consecuentemente, los motores de JavaScript usados en los navegadores actuales son exponencialmente más rápidos y funcionales que sus predecesores.

En su esencia, JavaScript es un motor de código script con las mismas características básicas que otros lenguajes de programación. Esto implica:

- Variables para almacenar información.
- Operadores para realizar cálculos y comparaciones.
- Funciones para agrupar instrucciones.
- Instrucciones condicionales y bucles para crear control de flujo.
- La habilidad de crear objetos con propiedades, métodos y eventos.

Por sí mismo, JavaScript no puede más que realizar cálculos y manipular datos, pero en combinación con el modelo DOM que implementan los navegadores puede hacer mucho más. Por ejemplo, podemos usar el código de JavaScript para:

- Añadir o quitar elementos de una lista mostrada en una página.
- Añadir, cambiar o quitar texto de una página.
- Cambiar los estilos CSS aplicados a conjuntos de elementos.
- Reaccionar a eventos, como un clic del ratón sobre un botón.
- Validar el contenido de un formulario antes de que sea enviado al servidor.
- Obtener información sobre el navegador, su fabricante y versión, e información del entorno como el tamaño de la ventana de Windows o la fecha actual.
- Mostrar una alerta en el navegador.

Además, la combinación de JavaScript y el API AJAX permite a las páginas hacer solicitudes asíncronas hacia el servidor web. El código JavaScript de una página puede usar esta característica para consultar a un servidor por información, sin tener que recargar la página.

Nota: JavaScript no es Java. Originalmente se denominaba Mocha y después LiveScript, antes de que su creador Brendan Eich, fichase por la compañía Sun y decidiese renombrarlo como JavaScript. El nombre estándar para este lenguaje es ECMA-262 porque Sun no licencia el nombre JavaScript. La implementación de Microsoft de ECMA-262 se llama JScript.

1.1.2. Sintaxis de JavaScript.

JavaScript tiene una sintaxis sencilla para escribir instrucciones, declarar variables y añadir comentarios.

Una instrucción es una simple línea de JavaScript. Representa una operación para ser ejecutada. Por ejemplo, la declaración de una variable, la asignación de un valor, o la llamada a una función.

El siguiente fragmento de código muestra algunos ejemplos de instrucciones:

```
var thisVariable = 3;  
counter = counter + 1;  
GoDoThisThing();
```

Todas las instrucciones de JavaScript deberían ser escritas en una única línea y terminar con un punto y coma. La excepción a esta regla es que podemos romper un string muy grande en varias líneas (por legibilidad) usando la barra diagonal. Por ejemplo:

```
document.write("Una increíble realidad \
```

es lo grande que es el mundo");

Nota: El finalizador de punto y coma actualmente es opcional. Sin embargo, si no terminamos la instrucción con punto y coma, JavaScript intenta discernir si debería estar puesto, algunas veces con resultados inesperados.

Podemos combinar instrucciones en bloques, delimitados entre llaves. Esta sintaxis es usada por funciones, bloques `if`, `while` o `for`.

Al igual que en otros lenguajes podemos usar comentarios para añadir notas descriptivas y documentación a nuestro código JavaScript. El intérprete JavaScript los ignora. JavaScript soporta dos estilos diferentes de comentar: comentarios de varias líneas que empiezan con `/*` y finalizan con `*/`, y comentarios de una sola línea que empiezan con `//` y finalizan con la línea.

Los comentarios deben describir el propósito o las razones de nuestro código, de forma que permita a otros programadores comprenderlo rápidamente.

```
document.write("Estoy aprendiendo JavaScript"); // muestra un mensaje al usuario
/* Podemos usar comentarios de varias líneas
   para añadir más información */
alert("¡Yo también estoy aprendiendo JavaScript!");
```

1.2. Variables, tipos de datos y operadores.

1.2.1. Variables.

Se usan variables para almacenar datos. En JavaScript hay varias formas de declarar una variable:

1. Dando un nombre y un valor. En este caso siempre se considerará la variable como global.
`saludo = "Hola";`
2. Declarándola sin dar un valor usando la palabra clave `var` o `let`. Hasta que la variable es asignada a un valor, JavaScript retornará su valor como indefinido (`undefined`).
`let misterio;`
3. Combinando ambas formas (que es el estilo más recomendado).
`var codigo = "Spang";`
4. Declarándola una constante con la palabra clave `const` y un valor.
`const IVA = 0.21;`
~~`IVA = 0.2;`~~ // ERROR

Una variable constante no puede cambiar de valor. Cualquier intento de hacerlo provocará un error.

Hay dos reglas importantes para poner nombre a las variables en JavaScript:

1. Los nombres de variable deben comenzar con una letra o un guion de subrayado.
2. Los nombres de variables son sensibles a mayúsculas y minúsculas.

Con esto en mente, se debe evitar dar nombres similares a variables que se diferencien en una sola letra, como `valorDeLeche` y `ValorDeLeche`. Se recomienda utilizar nombres descriptivos que hagan que el código sea más comprensible y fácil de depurar. Por ejemplo, `fechaSeleccionada`, `discoPreferido`, `sesionActual`.

1.2.2. Tipos de datos.

A diferencia de C#, Visual Basic y otros lenguajes habituales de programación, no podemos especificar el tipo de una variable en JavaScript. Se declara una variable con la palabra clave `var`, y entonces JavaScript intenta discernir el tipo de la variable según lo que le asignemos. JavaScript reconoce tres tipos simples:

1. Cadenas (**String**): Cualquier secuencia de caracteres encerrados entre comillas dobles o simples. Se utilizan caracteres de escape con la barra diagonal para los caracteres como `\` (doble comilla), `'` (comilla simple), `\,` (punto y coma), `\\` (barra diagonal), `\&` (ampersand). También se usa una barra diagonal para partir un string en varias líneas.

```
var simple = "Juan Pérez y José";
var escape = "\"Juan Pérez & José\"";
var muyLargo = "Este texto es demasiado \
largo para una sola línea.";
```

2. Números (**Number**): Cualquier número entero o decimal. Si se encapsula un número entre comillas dobles será tratado como un string.

```
var valor = 42;
var valorString = "42"; // es tratado como un string
```

3. Booleano (**Boolean**): Un valor lógico **true** (verdadero) o **false** (falso).

```
var esUtil = true;
```

JavaScript también convierte automáticamente datos entre tipos, lo cual puede llevar a confusión si no somos cuidadosos. Por ejemplo, el valor numérico **0** se avalúa a **false** en expresiones booleanas, así que es importante usar el operador correcto cuando comparamos los valores de variables. En todo caso podemos forzar conversiones entre datos:

```
let numero = Number("234"); // numero = 234
numero = Number("abc"); // numero = NaN
let texto = String(123); // texto = "123";
let b = Boolean(""); // b = false
b = Boolean("true"); // b = true
b = Boolean("false"); // b = true
b = Boolean(0); // b = false
b = Boolean(-1); // b = true
```

Un string vacío se considera **false** y cualquier otro string **true**. El número cero se considera **false** y cualquier otro número **true**.

Recordemos que, si declaramos una variable sin asignarle valor, la variable queda indeterminada. Por ejemplo:

```
let dato;
console.log(dato) // Imprime: undefined
```

Pero el valor **undefined** no es el equivalente a clásico valor **null**. Podemos asignar explícitamente el valor **null** a una variable:

```
let variableConValorNull = null;
```

Asignar una variable a **null** indica que su valor no existe, en vez de que no tiene asignado un valor. Es importante notar la diferencia.

Podemos determinar el tipo actual de un dato en una variable usando el operador **typeof**:

```
let dato = 99;
...
console.log(typeof dato) // Imprime: number
```

1.2.3. Operadores.

Un operador es una palabra clave o un símbolo que indica cómo combinar uno o más valores dentro de una expresión. Por ejemplo, el operador de suma **+** indica que dos números deben ser sumando en uno solo. Hay seis grupos de operadores en JavaScript:

1) Los operadores aritméticos, indican una función matemática:

+	suma
-	resta o signo de negación
*	multiplicación
/	división real
%	módulo (resto de la división entera)
++	incremento en una unidad
--	decremento en una unidad

Estos operadores funcionan de la manera esperada y no se diferencian de los usados en otros lenguajes.

```
let a = 7;
let b = 5;
let z = a + b; // z = 12
z++; // z = 13
--z; // z = 12
z = -a; // z = -7
z = a - b; // z = 2
z = a * b; // z = 35
z = a / b; // z = 1.4
z = a % b; // z = 1 (donde 7/5 produce el cociente 1 y el resto 2)
```

Simplemente debemos tener en cuenta que **0/0** retornará el valor especial **NaN** (no es un número), y que cualquier otro número dividido por cero retornará el valor espacial **Infinity**. Los operadores de incremento **++** o decremento **--** no funcionan igual si van antes o después de la variable:

```
let a = 7;
let b = ++a + 5; // b = (a=a+1) + 5 = (a=8) + 5 = 8 + 5 = 13, donde queda a = 8
a = 7;
b = a++ + 5; // b = (a) + 5 = 7 + 5 = 12, donde después queda a = a + 1 = 7 + 1 = 8
```

2) Los operadores de asignación, asignan valores a variables:

```
x = y
x += y (x = x + y)
x -= y (x = x - y)
x *= y (x = x * y)
x /= y (x = x / y)
x %= y (x = x % y)
```

También son análogos a los usados en Java, C#, Python, etc. Por ejemplo, con el operador `+=` asignamos a una variable el resultado de sumar la expresión de la derecha con el valor actual de la variable; lo que se conoce como una acumulación.

3) Los operadores de comparación, determinan si dos valores son iguales o no. El siguiente conjunto de operadores de comparación convierten los dos valores al mismo tipo antes de la comparación:

```
== (es igual a)
!= (no es igual a)
> (es mayor que)
< (es menor que)
>= (es mayor o igual que)
<= (es menor o igual que)
```

Este segundo grupo de operadores de comparación no convierte los dos valores al mismo tipo antes de la comparación:

```
=== (son igual en valor y del mismo tipo)
!== (no son iguales en valor o son de tipos distintos)
```

4) Los operadores booleanos, se usan para realizar operaciones lógicas:

```
x && y retorna true si x e y son ambos true, y sino retorna false.
x || y retorna true si tanto x como y son true, y sino retorna false.
!x retorna true si x es false, y false en otro caso.
```

5) El operador condicional ternario:

`?:` asigna uno de dos valores a una variable dependiendo de una condición. Por ejemplo, la expresión `x=(condicion)? valor1 : valor2;` asigna `x` a `valor1` si `condicion` es `true`, y a `valor2` en otro caso.

6) El concatenador de strings:

`+` concatena dos strings. Por ejemplo, `"Bo" + "om"` retorna `"Boom"`.

Hay una serie de cuestiones que debemos tener en cuenta con respecto a los operadores JavaScript y cómo convierten valores entre tipos cuando el intérprete JavaScript ejecuta las expresiones.

- Si concatenamos un número con un string, el resultado es un string.

```
x = 10 + 10; // x es asignado al número 20;
y = 10 + "10"; // y es asignado al string "1010";
z = "Ten" + 10; // x es asignado al string "Ten10";
```

- Siempre que queramos comparar datos teniendo en cuenta su tipo debemos usar el operador `===`. Por ejemplo, si declaramos las siguientes variables:

```
var x = "10";
var y = 10;
```

Obtendremos el siguiente resultado:

```
var b1 = x == y; // b1 = true
var b2 = x === y; // b2 = false
```

Con el operador `==` se realizan conversiones de tipo antes de la comparación, y con el operador `===` no.

- `0` (cero), `""` (el string vacío), `undefined` y `null` son evaluados a `false` en expresiones booleanas. Siempre se debería usar `===` cuando comparemos cualquiera de estos valores.

1.3. Instrucciones condicionales.

Las instrucciones condicionales permiten tomar decisiones y ejecutar diversas instrucciones dependiendo de si una condición booleana es verdadera o falsa. Hay varios tipos de instrucciones condicionales.

1.3.1. Instrucciones if/else.

Una instrucción `if` ejecuta un bloque de código si una condición es verdadera. Por razones sintácticas se debe encerrar la condición entre paréntesis. Por ejemplo:

```
let cantidadTotal = Number(prompt("La cantidad total"));
let pagoAdelantado = Number(prompt("El pago por adelantado"));

if (cantidadTotal > pagoAdelantado) {
  generarNuevoPrecio(); // se ejecuta si la condición es verdadera
}
```

Una instrucción `if` puede tener una cláusula `else` opcional que ejecuta un bloque de código si la condición del `if` es falsa.

```
if (cantidadTotal > pagoAdelantado) {
  generarNuevoPrecio(); // se ejecuta si la condición es verdadera
} else {
  generarFactura();      // se ejecuta si la condición es falsa
}
```

Podemos anidar instrucciones `if/else` entre sí:

```
let pagoMinimo = Number(prompt("El pago mínimo"));

if (cantidadTotal > pagoAdelantado) {
  generarNuevoPrecio();
} else {
  if (cantidadTotal < pagoMinimo) {
    error();
  } else {
    generarFactura();
  }
}
```

Aunque en estos casos se prefiere la siguiente sintaxis:

```
if (cantidadTotal > pagoAdelantado) {
  generarNuevoPrecio();
} else if (cantidadTotal < pagoMinimo) {
  error();
} else {
  generarFactura();
}
```

1.3.2. Instrucción switch.

Una instrucción `switch` permite realizar una serie de comparaciones sobre una expresión y ejecutar unas instrucciones determinadas si la expresión casa con un valor concreto. La expresión por comparar debe ser encerrada entre paréntesis después de la palabra clave `switch`, seguida de llaves. Dentro de las llaves se añaden cláusulas `case` seguidas de valores.

```
let tipoHabitacion = prompt("Tipo de habitación");
let precioHabitacion;
switch (tipoHabitacion) {
  case "Suite":
    precioHabitacion = 500;
    break;
  case "Real":
    precioHabitacion = 400;
    break;
  default: // si no casa ningún valor
    precioHabitacion = 300;
}
```



```
    break;
}
```

Si la expresión entre paréntesis casa con el valor que va después de un **case**, se ejecutarán todas las instrucciones siguientes hasta llegar al final del bloque del **switch** o hasta encontrar una instrucción **break**. Si la expresión no casa con ningún valor se ejecutará la cláusula **default**.

Es importante el uso de la instrucción **break**, la cual provoca que el código se siga ejecutando después de la llave de cierre de **switch**. Sin el **break**, al entrar por un **case** se ejecutarán todas las instrucciones que se encuentren hasta llegar al final del **switch**.

Nótese en este ejemplo, que, si el valor de **tipoHabitacion** es "suite" o "real", la variable **precioHabitacion** será asignada a 300, porque la comparación es sensible a mayúsculas y minúsculas. Para solventar esto, podemos convertir el valor de **tipoHabitacion** a minúsculas usando:

```
switch (tipoHabitacion.toLowerCase()) {
  case "suite":
    ...
  case "real":
    ...
}
```

La función **toLowerCase()** puede ser aplicada sobre cualquier variable que contenga un string, y retorna su valor convertido a minúsculas. Si se aplica sobre una variable que no contenga un string provocará un error.

Las reglas para tener en cuenta con **switch** son las siguientes:

- La expresión entre paréntesis puede ser cualquier tipo de dato.
- Después de la cláusula **case** podemos poner valores literales, variables o expresiones.
- Si se repiten valores en la cláusula **case** se ejecutará sólo la primera que **case**.
- Se utilizan instrucciones **break** para parar la ejecución de un bloque **case** y evitar que se ejecuten los bloques de los **case** siguientes.

1.3.3. Expresión condicional.

También podemos utilizar el operador ternario como instrucción condicional:

```
let esmayor = 4 > 3 ? "si" : "no";
```

Es operador es útil cuando debemos utilizar un **if/else** para realizar asignaciones sobre una misma variable. En vez de escribir:

```
let edad = Number(prompt("Tu edad"));
let mensaje;
if (edad < 18) {
  mensaje = "Eres menor de edad";
} else {
  mensaje = "Eres mayor de edad";
}
```

Resulta más sintético esto:

```
mensaje = edad < 18 ? "Eres menor de edad" : "Eres mayor de edad";
```

1.4. Bucles.

Los bucles o instrucciones de iteración permiten iterar varias veces sobre un conjunto de instrucciones hasta que deje de cumplirse una condición. Hay varios tipos de bucles en JavaScript.

1.4.1. Bucles while.

Un bucle **while() { }** evalúa una condición booleana, y ejecuta el bloque de código repetidamente mientras la condición sea cierta. Si la condición es inicialmente falsa el bloque de código no se ejecuta ninguna vez. Por ejemplo:

```
let year = prompt("Tu año de nacimiento");
let age = 0;
while ( year < 2021 ) {
  age ++;
  year ++;
}
```

Con este bucle conseguimos obtener la edad (**age**) de alguien hasta el año 2021. Si año introducido por el usuario es el propio 2021 el bucle no se repetirá ninguna vez y obtendremos edad cero.

Un bucle `do{ } while ( )` ejecuta un bloque de código y evalúa una condición booleana para decidir si repite el bloque o no. Si la condición es verdadera se vuelve a ejecutar el bloque de código, y la condición vuelve a ser reevaluada. El bloque se ejecuta por lo menos una vez. Por ejemplo:

```
let year = prompt("Tu año de nacimiento");
let age = 0;
do {
  age++;
  year++;
} while ( year < 2021 );
```

Con este bucle, si año introducido por el usuario es el propio 2021 el bucle se ejecutará una vez y obtendremos edad 1.

1.4.2. Bucle for.

Un bucle `for(;;)` ejecuta un bloque de código mientras una condición es verdadera. Esta condición se basa normalmente en el valor de una variable de control. La ejecución del bucle `for` es determinada por tres segmentos separados por punto y coma: instrucciones de inicialización, una condición de finalización e instrucciones de paso. Por ejemplo:

```
let year = prompt("Tu año de nacimiento");
let age;
for (age = 0; year < 2020; age++, year++) {
}
```

En el primer segmento del `for` inicializamos la variable `age` puesto que este segmento se ejecuta una única vez. En el segundo segmento se pone la condición del bucle (al igual que con `while`). En cada repetición o iteración se evalúa la condición, y si es verdadera se ejecuta el bloque de código y después se ejecutan las instrucciones del tercer segmento del `for`.

Tanto en el primer segmento podemos poner varias instrucciones válidas separadas por comas. Pero en el segmento central sólo se puede poner una única expresión booleana (simple o compuesta con los operadores `&&` y `||`).

Un bucle `for( in )` ejecuta un bloque de código tantas veces como se recorre un dato iterable. Un dato es iterable si consta de una secuencia de elementos, de forma que el bucle se repetirá tantas veces como elementos. En JavaScript son iterables los strings y en general cualquier objeto.

En el caso de los string podemos usar este bucle para recorrer sus caracteres:

```
let texto = "Hola";
for (let i in texto) {
  console.log( texto.charAt( i ));
}
```

La variable `i` tomará valores desde cero hasta la longitud del string menos 1. El método `charAt()` de los string permite recuperar uno de sus caracteres por posición.

Veremos cómo usar este método sobre objetos y en especial sobre arrays.

1.4.3. Forzar el comportamiento de los bucles.

Todos los bucles permiten controlar su ejecución mediante las instrucciones `break` y `continue`.

Una instrucción `break` permite romper la ejecución del bucle desde dentro del bloque de código. Cuando la ejecución llega a la instrucción `break`, el bucle para y continúa la ejecución en la siguiente instrucción fuera del bucle.

```
for (let i=0; i<10; i++) {
  if (i == 6) {
    break; // se rompe el bucle cuando la variable i sea igual a 6
  }
  console.log(i);
}
```

Por su parte, con la instrucción `continue` podemos parar la ejecución en una iteración y pasar inmediatamente a la siguiente:

```
for (let i=0; i<10; i++) {
  if (i == 6) {
```



```

    continue; // deja de ejecutar la iteración actual y pasa a la siguiente
  }
  console.log(i);
}

```

Con este código se mostrarán valores de 0 a 9 pero saltándose el 6.

Cuando hay varios bucles anidados, las instrucciones **break** y **continue** afectan al bucle que las contiene.

```

for (let i=0; i<10; i++) {
  for (let j=0; j<10; j++) {
    if (i == 6)
      break; // finaliza el bucle de la j pero no el de la i
  }
}

```

Podemos etiquetar cada bucle e indicar a cuál debe afectar **break** o **continue**:

```

bucle1: for (let i=0; i<10; i++) {
  bucle2: for (let j=0; j<10; j++) {
    if (i == 6)
      continue bucle1; // pasa a la siguiente iteración del primer bucle
  }
}

```

1.5. Trabajando con arrays.

El tipo **Array** permite crear y trabajar con secuencias de valores posicionados en base cero. De forma que podemos añadirle o quitarle o modificar sus valores dinámicamente.

1.5.1. Creación de arrays.

Se pueden crear arrays de varias maneras:

```

// Arrays vacíos
array1 = Array();
array2 = new Array();
// Arrays inicializados con los valores 1, 2 y 3
array3 = Array(1, 2, 3);
array4 = new Array(1, 2, 3);

```

Aunque la preferida es la siguiente:

```

array5 = [];
array6 = [1, 2, 3];

```

Realmente el tipo **Array** es un objeto capaz de contener valores asociados a claves, más una secuencia de valores asociados a índices comenzando por cero. En la siguiente figura podemos ver una representación figurada de los datos simples, los objetos y los arrays:

DATO SIMPLE

```
let x = 34;
```

OBJETO

```

let cliente = {};
cliente.nombre = "Juan";
cliente.edad = 40;

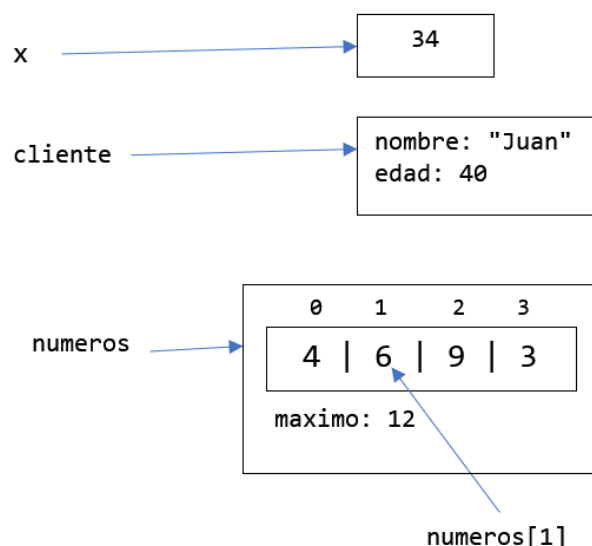
```

ARRAY

```

let numeros = [4, 6, 9, 3];
numeros.maximo = 12;

```



La secuencia que tiene asociada el Array esta formada por celdas numeradas desde cero. Se accede o cambia un valor de una celda indicando su posición entre corchetes:

```
numeros[0] += 2; // en la celda cero se cambia el 4 por un 6
numeros[3] = numeros[1] + numeros[2]; // en la celda 3 se pone 15
```

En todo momento podemos saber cuántas celdas hay consultado el atributo `length`:

```
console.log( numeros.length ); // Imprime: 4
```

Si utilizamos un Array como una lista secuencial, los índices enteros (contando desde cero) actúan como claves:

```
let numeros = [4, 6, 9, 3];
console.log( numeros[0] ); // Imprime: 4
console.log( numeros[2] ); // Imprime: 9
```

Si usamos un Array como un diccionario o mapa, podemos tener claves no numéricas asociadas a valores. Por ejemplo:

```
var personas = [ ];
personas["Juan"] = "López";
personas["Ana"] = "Gutierrez";
```

Aquí estamos usando el array de forma asociativa, asociamos una clave a un valor (en este caso un nombre a su apellido). Pero también podemos añadirle elementos indexados:

```
personas[0] = "Marta";
personas[1] = "Javier";
```

Si accedemos a los elementos del Array usando un bucle `for(;;)` obtendremos:

```
let texto = "";
for(let i=0; i<personas.length; i++) {
  texto += personas[i] + " ";
}
console.log( texto ); // Imprime: Marta Javier
```

Cómo sólo utilizamos las claves numéricas accedemos a la secuencia de celdas, no a los atributos. Pero si usamos un bucle `for( in )`:

```
let texto = "";
for(let i in personas) {
  texto += personas[i] + " ";
}
console.log( texto ); // Imprime: Marta Javier Juan Ana
```

Se recorre primero las posiciones de las celdas y después los atributos.

También podemos usar un Array como una pila. En una pila los elementos se colocan unos encima de otros, de forma que sólo se puede sacar el último elemento disponible. El método `push()` permite añadir un elemento al final del array, y el método `pop()` extraer el último elemento:

```
let lista = [ ];
lista.push("Ana");
console.log(lista.pop());
```

El método `push()` añade el nuevo elemento al final del array asignándole como índice la longitud del array. El método `pop()` recupera y extrae del array el último elemento.

1.5.2. Operaciones con arrays.

Para añadir elementos a un array podemos usar el método `push()` para añadir nuevas celdas al final, o el método `unshift()` para insertar nuevas celdas al inicio:

```
let personas = [ ];
personas.push( "Juan", "Ana"); // ["Juan", "Ana"]
personas.unshift( "Javier"); // ["Javier", "Juan", "Ana"]
```

También podemos añadir elementos usando un índice aún no utilizado. Para hacerlo al final podemos usar como índice la longitud al array:

```
personas[personas.length] = "Jose"; // ["Javier", "Juan", "Ana", "Jose"]
```

Si usamos un índice mayor que el número de elementos actuales se considerará que las celdas intermedias quedan vacías (valor `undefined`).

Para eliminar celdas podemos usar varios métodos:

- El método `pop()` elimina el último elemento, y lo retorna.

- El método `shift()` elimina el primer elemento, y lo retorna.
- El método `splice()` elimina un rango de elemento. Debemos indicar la posición inicial y el número de elementos a eliminar:

```
let personas = ["Javier", "Juan", "Ana", "Jose"];
personas.splice(1, 2); // ["Javier", "Jose"]
```

Si no se indica el tamaño se elimina desde la posición indicada hasta el final.

Para encontrar un elemento dentro del array se puede usar el método `indexOf()`. Este método retorna la posición de un elemento o -1 si no lo encuentra:

```
let index = personas.indexOf("Ana"); // index = 1
```

El método `lastIndexOf()` hace lo mismo que `indexOf()` pero buscando desde el final.

El método `sort()` permite ordenar el array:

```
let personas = ["Javier", "Juan", "Ana", "Jose"];
personas.sort(); // ["Ana", "Javier", "Jose", "Juan"]
```

Podemos concatenar varios arrays con el método `concat()`:

```
let personas = ["Javier", "Juan"];
personas = personas.concat(["Ana", "Jose"]); // ["Javier", "Juan", "Ana", "Jose"]
```

Pero este método retorna un nuevo array, sin modificar el original.

Y para obtener parte de un array se utiliza el método `slice()`:

```
let personas = ["Javier", "Juan", "Ana", "Jose"];
let trozo = personas.slice(0, 2); // trozo = ["Javier", "Juan"]
```

Se indica la posición inicial (que es incluida) y la posición final (que no es incluida). Si no se indica la posición final se obtiene el trozo desde la posición inicial hasta el final.

Otros métodos más complejos de los arrays serán analizados posteriormente.

1.6. Funciones.

Una función es una secuencia de instrucciones con nombre, que realiza alguna tarea específica. Las funciones se usan para definir bloques de código reutilizables. Después de que ha sido definida una función, puede ser invocada desde cualquier sitio del script y la secuencia de instrucciones que forman la función será ejecutada, y después de la ejecución se retornará a la instrucción siguiente a la de la llamada a la función.

1.6.1. Sintaxis de las funciones.

La definición de funciones en JavaScript tiene siempre la misma sintaxis:

```
function unNombre ( argumento1, argumento2, ..., argumentoN ) {
  instrucción1;
  instrucción2;
  ...
  instrucciónN;
}
```

Hay cuatro partes en la definición de una función:

- La palabra clave **function** indica que comienza la declaración de la función.
- El nombre de la función. Se usa para invocar la función, y es sensible a mayúsculas y minúsculas.
- Una lista de variables separadas por comas, llamadas parámetros, a través de los cuales podemos pasar valores al interior de la función. Aunque la función no tenga parámetros se deben dejar los paréntesis que delimitan la lista de parámetros. No hay que indicar el tipo de los parámetros, basta con poner un nombre válido de variable.
- Una serie de instrucciones encapsuladas entre llaves.

Las funciones de JavaScript pueden incluir o no cláusulas **return** para retornar valor. En todo caso si una función no retorna nada y se utiliza en un contexto donde queremos evaluar lo que retorna se obtendrá el valor **undefined**.

Como ejemplo, veamos una función que suma dos argumentos:

```
function suma(a, b) {
  return a + b;
}
```

En el cuerpo de la función se utiliza **return** para retornar el resultado de la suma de los parámetros **a** y **b**. Pero esta instrucción sólo se ejecutará cuando invoquemos la función pasándole dos valores concretos:

```
resultado = suma(4, 5); // La variable resultado queda asignado al valor 9
```

1.6.2. Funciones como dato.

Las funciones son también datos. En este caso, el dato consiste en un conjunto de instrucciones, y por tanto podemos asignarlas a variables:

```
function suma(a, b) {  
    return a + b;  
}  
let add = suma;
```

De hecho, el propio nombre con el cual declaramos una función es una variable. Podemos ver la diferencia entre imprimir la función como dato e imprimir su ejecución:

```
console.log( add );           // Imprime: f suma(a, b) { return a + b; }  
console.log( add(3, 4) );     // Imprime: 7
```

También podemos crear directamente una función sin nombre y asignarla a una variable. A continuación, se muestra un ejemplo:

```
let funcSaludo = function () {  
    alert("Hola...");  
};
```

Podemos usar ahora la variable `funcSaludo` para invocar la función:

```
funcSaludo();
```

Incluso podemos declarar una función anónima y ejecutarla sin asignarla a una variable:

```
(function () { alert("Hola..."); })();
```

Debemos encapsular la función anónima entre paréntesis y a continuación se invoca añadiendo los paréntesis de los argumentos.

La última especificación de JavaScript permite una sintaxis más simplificada para las funciones anónimas, las llamadas funciones flecha:

```
let funcSaludo = () => { alert("Hola..."); };
```

Aparentemente no parece que la sintaxis se haya acortado, pero ocurre que si la función tiene una única instrucción no hacen falta las llaves:

```
let funcSaludo = () => alert("Hola...");
```

Incluso si la función retorna un valor, no es necesario un `return`:

```
let suma = (a, b) => a + b;
```

Si la función tiene un único argumento se puede prescindir de los paréntesis:

```
presentarse = nombre -> alert("Soy " + nombre);
```

Si no tiene parámetros o tiene más de uno hay que poner los paréntesis.

1.6.3. Comportamiento de los parámetros.

Las funciones usan los parámetros como variables locales que reciben copias de los datos que se pasan como argumento cuando se invoca la función. Esto quiere decir que en las instrucciones de la función podemos modificar el valor de estos parámetros sin que afecte al dato pasado como argumento.

JavaScript aplica por defecto una declaración opcional de sus parámetros. Esto quiere decir que al invocar una función podemos pasar o no argumentos a todos los parámetros. Por ejemplo, podemos definir una función que sume varios números:

```
function suma(a, b, c) {  
    if (a===undefined) a = 0;  
    if (b===undefined) b = 0;  
    if (c===undefined) c = 0;  
    return a + b + c;  
}
```

Como no hay obligación de pasar valores a los parámetros aquellos que no reciben valor quedan a `undefined`. Por eso se realiza en el código la comprobación de que si no están definidos reciban el valor cero. Operar con el valor `undefined` devolvería el valor `NaN`.

Podemos invocar ahora esta función de varias maneras:

```
var resultado = suma(5, 7, 3);    // resultado = 15  
var resultado = suma(5, 7);       // resultado = 12  
var resultado = suma(5);          // resultado = 5  
var resultado = suma();           // resultado = 0
```

En realidad, podríamos incluso pasar más argumentos de los solicitados:

```
var resultado = suma(1, 2, 3, 4, 5); // resultado = 6
```

Y es que las funciones reciben todos los argumentos en una colección denominada **arguments**, independientemente de si declaran parámetros o no. Podemos crear la siguiente versión de la función **suma()**:

```
function suma() {
  var resultado = 0;
  for (var i = 0; i < arguments.length; i++) {
    resultado += arguments[i];
  }
  return resultado;
}
```

De esta manera, podemos invocar la función **suma()** con el número de argumentos que queramos, ya que todos serán sumados.

Para evitar las comprobaciones sobre parámetros sin valor, podemos declarar parámetros con valores por defecto:

```
function resta(a=0, b=0) {
  return a - b;
}
```

De esta forma podemos realizar las siguientes ejecuciones:

```
resultado = resta(); // resultado = 0
resultado = resta(4); // resultado = 4
resultado = resta(4, 3); // resultado = 1
```

2. Objetos.

Los objetos son el corazón del lenguaje JavaScript, puesto que casi todo es un objeto. Los objetos pueden contener datos, funciones (o métodos) y otros objetos.

En principio podemos ver un objeto como un dato compuesto de varios valores, donde cada valor interno se identifica con un nombre de atributo:

```
let objeto = { }; // un objeto vacío
objeto.cantidad = 6; // le metemos un valor simple
objeto.saludar = () => console.log("Hola"); // le metemos un método
objeto.valores = [1, 2, 3]; // le metemos otro objeto (un array)
```

2.1. Usando el espacio de nombres global.

Como en JavaScript todo gira en torno a objetos que contienen a otros objetos se define un objeto global que contiene todo lo demás. Este objeto se denomina **window** y se considera que cualquier declaración realizada con **var** fuera de una función se realiza dentro de este objeto:

```
<script>
  var nombre = "Juan";
  console.log( window.nombre ); // Imprime: Juan
</script>
```

Las declaraciones con **let** y **const** son siempre locales dentro de su ámbito y no forman parte de **window**.

Se considera a **window** como un espacio de nombres donde se pueden declarar nombres de variables, funciones, objetos, etc. En general, podremos usar cualquier nuevo objeto como un subespacio de nombres.

Para acceder a un nombre definido dentro de un espacio de nombres se utiliza el punto. En el ejemplo previo, la variable **nombre** se define por defecto dentro del objeto **window**, y por eso podemos acceder a él con **window.nombre**. El objeto predefinido **console** también pertenece a **window**, pero define su propio espacio de nombres que contiene un método **log()**.

Este concepto de espacio de nombres es algo implícito y normalmente no debe preocuparnos, pero resulta esencial para comprobar si realmente existe o no una variable. Por ejemplo, si ejecutamos el siguiente código:

```
<script>
  if ( precio !== undefined ) {
    console.log( precio );
  }
</script>
```

Obtendremos un error, puesto que la variable precio aún no está definida. Sin embargo, esto sí funcionará:

```
<script>
  if ( window.precio !== undefined) {
    console.log( precio );
  }
</script>
```

El objeto `window` siempre existe y simplemente comprobamos que contiene una variable.

2.2. Creación de objetos.

Hay varias formas de crear objetos vacíos:

```
let obj1 = new Object();
let obj2 = Object();
```

Pero la preferida es la siguiente:

```
let obj3 = { };
```

Una vez creado un objeto podemos añadir nombres a su espacio asignando valores:

```
obj3.texto = "Hola";
```

Los nombres utilizados después del punto se denominan atributos del objeto. Podemos crear también un objeto con atributos:

```
let obj4 = { 'valor': 4, 'texto': 'hola' };
let obj5 = { valor: 4, texto: 'hola' };
```

Los nombres de atributos pueden ir o no con comillas, pero los valores después de dos puntos deben escribirse con su sintaxis correcta.

Cada objeto debería representar los datos y operaciones de un concepto bien definido. Por ejemplo, si en nuestra aplicación tenemos que gestionar productos podemos crear objetos producto con sus datos:

```
let producto1 = { nombre: 'portatil', preciobruto: 500 }
```

Como los objetos también pueden contener funciones sería interesante añadir un método que retorne el precio de venta al público aplicando un IVA general:

```
const IVA = 0.2;
let producto1 = { nombre: 'portatil', preciobruto: 500, pvp: () => this.preciobruto + (1*IVA) }
```

Para que se vea más claramente lo que hemos hecho vamos a ponerlo así:

```
const IVA = 0.2;
let producto1 = { };
producto1.nombre = 'portatil';
producto1.preciobruto = 500;
producto1.pvp = function () {
  return this.preciobruto + (1*IVA);
}
```

Dentro del método `pvp()` se usa `this.preciobruto` para recuperar el precio del producto y aplicarle el IVA. La palabra clave `this` representa el objeto actual, en este caso `producto1`. El código funcionaría igual si ponemos:

```
producto1.pvp = function () {
  return producto1.preciobruto + (1*IVA);
}
```

Pero si queremos crear otro producto con los mismos atributos tendríamos que ir adaptando esta referencia. Es más simple usar `this` y copiar todo el código de los métodos para otro objeto del mismo tipo:

```
let producto2 = { nombre: 'ratón', preciobruto: 20, pvp: () => this.preciobruto + (1*IVA) }
```

Aun así, si tenemos que crear muchos objetos producto será muy engorroso tener que repetir el código de los métodos. Para simplificar esto se hará uso de plantillas donde definiremos todos los atributos comunes para varios objetos, y después crearemos los objetos a partir de estas plantillas.

2.3. Creando objetos usando funciones constructoras.

JavaScript incluye varias formas de crear plantillas para objetos. Aquí vamos a ver la forma clásica usando una función constructora.

Una función constructora es cualquier función definida con la palabra clave `function`. Dentro de estas funciones se utiliza la palabra clave `this` para hacer referencia a un objeto que se va a crear.

Como ejemplo vamos a definir una función constructora para crear objetos de tipo cuenta bancaria:


```
const Cuenta = function (id = 0, nombre='') {
  this.id = id;
  this.nombre = nombre;
  this.balance = 0;
  this.movimientos = [ ]; // contendrá algo como [{operacion: 'ingreso', cantidad: 100}]
  this.ingresos = () => this.movimientos.filter( m => m.operacion==='ingreso' );
  this.reintegros = () => this.movimientos.filter( m => m.operacion==='reintegro' );
};
```

Dentro de la función constructora se declaran atributos en el objeto predefinido **this**, y podemos usar los parámetros de la función para asignar datos iniciales.

Después de definir la función constructora podemos usarla para crear un nuevo objeto usando la palabra clave **new**. Podemos pasar opcionalmente argumentos a la función constructora para especificar los valores iniciales del objeto. Cuando creamos un objeto usando **new**, JavaScript realiza los siguientes pasos:

1. Crea un nuevo objeto.
2. Asigna **this** a una referencia del nuevo objeto.
3. Asigna los parámetros de la función para el nuevo objeto.

El siguiente ejemplo creas dos objetos **Cuenta** y asigna sus propiedades:

```
let acc1 = new Cuenta(1, "Juan");
let acc2 = new Cuenta(2, "María");
```

Podemos añadir ingresos y reintegros:

```
acc1.movimientos.push( {operacion: 'ingreso', cantidad: 100});
acc1.balance += 100;
acc1.movimientos.push( {operacion: 'reintegro', cantidad: 50});
acc1.balance -= 100;
console.log( acc1.ingresos() );
console.log( acc1.reintegros() );
```

Ahora bien, este modo de trabajar con estos objetos queda un poco inconsistente. Debemos añadir movimientos a un array y además actualizar el balance. Si no actualizamos el balance no será consistente con los movimientos añadidos. Todo debería ser más simple.

Vamos a incluir métodos para ingresar y reintegrar, y ocultaremos el acceso a los atributos que controlan los movimientos y cifras.

```
const Cuenta = function (id = 0, nombre='') {
  this.id = id;
  this.nombre = nombre;
  let _balance = 0; // Defino una variable local
  let _movimientos = [ ]; // Defino una variable local
  this.ingresar = (cantidad) => {
    _movimientos.push( {operacion: 'ingreso', cantidad: cantidad} );
    _balance += cantidad;
  };
  this.reintegrar = (cantidad) => {
    _movimientos.push( {operacion: 'reintegro', cantidad: cantidad} );
    _balance -= cantidad;
  };
  this.balance = () => _balance;
  this.ingresos = () => _movimientos.filter( m => m.operacion==='ingreso' );
  this.reintegros = () => _movimientos.filter( m => m.operacion==='reintegro' );
};
```

Las variables locales dentro de una función se pueden usar sin ningún problema en cualquier instrucción dentro de la función, pero no son visibles fuera de ella. Con esto conseguimos que el uso de estos objetos sea más comprensible y evitamos inconsistencias:

```
acc1.ingresar(100);
acc1.reintegrar(50);
console.log( acc1.balance() );
console.log( acc1.ingresos() );
```

```
console.log( acc1.reintegros() );
```

A veces bastará con crear un único objeto de su tipo (lo que se conoce como un objeto singleton), pero sigue siendo interesante utilizar una función constructora para encapsular todo el código relacionado con ese objeto. Por ejemplo, si en un programa vamos a gestionar productos a los cuales se les aplica un IVA general, todo puede ir dentro de un objeto singleton de la siguiente manera:

```
const Almacen = new function () {
  this.Producto = function (nombre = "", precio = 0) {
    this.nombre = nombre;
    this.precio = precio;
  }

  this.IVA = 0.20;
  this.productos = [];
  this.precioTotal = function () {
    let total = 0;
    this.productos.forEach(p => total += p.precio * (1+this.IVA));
    return total;
  }
}

Almacen.productos.push( new Almacen.Producto("portatil", 500));
Almacen.productos.push(new Almacen.Producto("ratón", 20));
console.log(Almacen.precioTotal());
```

Declarando **Almacen** como una constante y aplicando **new** directamente sobre la función constructora conseguimos un único objeto de esta plantilla. Y dentro de la plantilla metemos todo.

Lo habitual es que la función constructora para **Producto** se declare por separados, pero en este ejemplo se ha hecho así para ver que dentro de una función constructora podemos declarar cualquier cosa.

2.4. Elementos HTML como objetos.

Todos los navegadores actuales implementan un modelo de objetos llamado DOM donde se incluye un objeto por cada elemento HTML de la página. Estos objetos tendrán un atributo por cada atributos de la etiqueta HTML.

2.4.1. Encontrar elementos en DOM.

Después de que una página ha sido cargada, una acción común es encontrar un elemento o un conjunto de elementos para consultarlo o manipularlo. Por ejemplo, podemos necesitar obtener una referencia a una lista para poblarla con elementos recuperados de un servicio web.

DOM representa varias partes de un documento como un conjunto de arrays:

- El array **forms** contiene todos los formularios del documento (elementos FORM).
- El array **images** contiene todas las imágenes del documento (elementos IMG).
- El array **links** contiene todos los hiperenlaces del documento (elementos A).
- El array **anchors** contiene todas las etiquetas **<a>** del documento con un atributo **name**.

Todas estas colecciones son atributos del objeto **document**. Este objeto representa al documento. Podemos usar la notación del punto para acceder a cada colección, a cualquier atributo, método o evento definido en DOM. Para acceder a elementos individuales de una colección, se puede usar un índice o una referencia al valor del atributo **name**. Por ejemplo, si tenemos el siguiente formulario en una página:

```
<form name="contactForm">
  <input type="text" name="nameBox" id="nameBoxId" />
</form>
```

Podemos acceder al objeto DOM que representa al formulario de las siguientes formas:

```
document.forms[0]           // forms es un array en base cero
document.forms["contactForm"]
document.forms.contactForm
document.contactForm
```

También se puede acceder al objeto DOM que representa la caja de texto dentro del formulario de varias formas, incluyendo las siguientes:

```
document.forms.contactForm.elements[0]
document.forms.contactForm.elements["nameBox"]
document.forms.contactForm.nameBox
document.contactForm.nameBox
```

Alternativamente DOM define métodos en el objeto **document** que recuperan elementos según el valor de sus atributos **ID** y **name**. Podemos usar estos métodos para encontrar rápidamente elementos determinados, sin tener que recorrer toda la ruta de dependencias de los elementos. Estos métodos incluyen:

- **document.getElementById(idString)**, que retorna el primer elemento cuyo atributo **id** coincide con el argumento.
- **document.getElementsByName(nameString)**, que retorna un array de elementos cuyos atributos **name** coinciden con el argumento.
- **document.getElementsByClassName(classString)**, que retorna un array de elementos cuyos atributos **class** coinciden con el argumento.
- **document.getElementsByTagName(nameTagString)**, que retorna un array de elementos cuyo nombre de etiqueta coincide con el argumento.

El siguiente ejemplo muestra cómo obtener una referencia al cuadro de texto **nameBoxId** del formulario usando su **id** y su **name**:

```
var userNameBox = document.getElementById("nameBoxId");
var username = userNameBox.value;
userNameBox = document.getElementsByName("nameBox")[0];
username = userNameBox.value;
```

También podemos buscar elementos usando selectores CSS mediante el método **querySelector()**. Este método retorna el **primer** objeto que cumple con el selector:

```
var userNameBox = document.querySelector("input[type=text]");
```

2.4.2. Modificación de elementos.

Una vez que tenemos la referencia de un elemento DOM podemos acceder o modificar sus atributos. Al modificar un atributo mediante código JavaScript los cambios visuales se reflejarán inmediatamente en la página.

En el DOM, todos los elementos HTML se definen como objetos, y estos objetos poseen atributos y métodos. Aquí vamos a hacer referencia a atributos y métodos que nos permitirán cambiar el aspecto de un elemento DOM en la página.

El atributo **innerHTML** hace referencia al contenido dentro de un elemento. Aunque no todos los elementos permiten contenido, podremos modificar el contenido de elementos **DIV**, **P**, **SPAN**, **LABEL**, etc.

El siguiente ejemplo nos permite cambiar el contenido de texto de un párrafo:

```
<body>
  <p id="parrafo1"></p>

  <script>
    var parrafo1 = document.getElementById("parrafo1");
    parrafo1.innerHTML = "Saludos.";
  </script>
</body>
```

Pero modificando este atributo sobre etiquetas contenedoras de otros elementos podemos realizar acciones dinámicas para agregar o quitar elementos en tiempo de ejecución. Por ejemplo, el siguiente código contiene un botón y un elemento **<div>**. Al pulsar el botón se añade dinámicamente una lista dentro del **div**:

```
<input type="button" onclick="modificar('div1')" value="modificar" />
<div id="div1"></div>

<script>
  function modificar(elemento) {
    var item = document.getElementById(elemento);
    item.innerHTML += "<ul><li>Uno</li><li>Dos</li></ul>"
  }
</script>
```

Para ocultar o mostrar un elemento podemos usar el atributo **hidden**:

```
function changeHidden(elemento) {
```

```

    var item = document.getElementById(elemento);
    item.hidden = ! item.hidden;
}

```

Y en general podemos cambiar cualquier atributo de una etiqueta de dos formas: por su nombre, o mediante el método `setAttribute()`:

```
<div id="div1"></div>
```

```

<script>
    var item = document.getElementById("div1");
    item.name = "div 1";
    item.setAttribute("name", "div 1");
</script>

```

Para modificar los estilos CSS de un elemento podemos usar el atributo `style`. Por ejemplo, la siguiente función asigna un estilo CSS a un elemento:

```

<script>
    function cssChange(elemento, propiedad, valor) {
        var item = document.getElementById(elemento);
        item.style[propiedad] = valor;
    }

```

```

    cssChange("div1", "background-color", "yellow");
</script>

```

2.4.3. Manejando eventos en DOM.

HTML define varios eventos desencadenados por el usuario que podemos asociar a una acción JavaScript. Por ejemplo, cuando un usuario **mueve** el ratón por encima de un elemento, se lanza el evento `mouseover`. Cuando se lanza un evento, si hemos asignado un oyente para el evento, el navegador lo ejecutará. Muchos elementos HTML proporcionan atributos que comienzan por `"on"` seguido del nombre de un evento. En estos atributos podemos asignar funciones o código JavaScript. Por ejemplo, podemos manejar el evento `mouseover` de un elemento `<img>` usando el atributo `onmouseover`, tal como sigue:

```

```

2.5. Objetos de utilidad.

Además del objeto global `window`, JavaScript define otros contenidos en su espacio de nombres:

- **document**: representa la página actual y proporciona métodos de utilidad para acceder a los elementos de la página.
- **navigator**: representa al navegador que ha cargado la página, y ofrece métodos de utilidad para interactuar con él.
- **console**: representa la consola de depuración que incluyen los navegadores.
- **Math**: contiene atributos y métodos para operaciones aritméticas y trigonométricas.
- **JSON**: contiene métodos para convertir objetos a formato Json y viceversa.

Hay muchos más que no vamos a explorar ahora.

2.5.1. Funciones globales.

El objeto global `window` proporciona algunas funciones de interés:

- **alert(mensaje)**: muestra un cuadro de mensaje.
- **prompt(mensaje)**: solicita un dato al usuario mostrando un mensaje.
- **isNaN(value)**: retorna true si el valor dado no es un número.
- **parseInt(value, base)**: convierte un dato a un número según una base numérica. Por ejemplo, vamos a convertir un string que contiene un número en base 8:

```
console.log( parseInt("123", 8) ); // Imprime: 83
```

2.5.2. Operaciones matemáticas.

El objeto singleton `Math` contiene las constantes y funciones matemáticas habituales.

```

console.log(Math.PI);
console.log(Math.E);

```

Proporciona funciones trigonométricas como el coseno, seno, tangente, arcocoseno, etc. El argumento de estas funciones se interpretará en radianes:

```
let c = Math.cos( Math.PI );
```

También tenemos funciones de redondeo:

```
c = Math.floor( 32.6);           // c = 32   redondea siempre hacia abajo
c = Math.ceil( 32.6);           // c = 33   redondea siempre hacia arriba
c = Math.round( 32.6);          // c = 33   redondea según los decimales
c = Math.trunc( 32.6);          // c = 32   elimina los decimales
```

Operaciones aritméticas:

```
c = Math.pow(10, 3);             // c = 1000  potencia de 10 elevado a 3
c = Math.min( 4, 6 );           // c = 4     valor mínimo
c = Math.max( 4, 6 );           // c = 6     valor máximo
c = Math.log(Math.E);           // c = 1     logaritmo en base E
c = Math.log10(10);             // c = 1     logaritmo en base 10
```

Números aleatorios:

```
c = Math.random();              // un número en el rango semiabierto [0, 1)
```

Si queremos obtener valores aleatorios enteros podemos aplicar las siguientes operaciones. Para obtener un número en el rango [0, 10):

```
let r = Math.trunc(Math.random() * 10);
```

Para obtener un número en el rango [2, 20):

```
let r = Math.trunc(Math.random() * 18) + 2;
```

En general, para obtener un número entre un mínimo inclusive y un máximo no inclusive tendremos:

```
function random(min, max) {
  return Math.trunc(Math.random() * (max - min)) + min;
}
```

2.5.3. El formato Json.

El formato Json surgió con la necesidad de representar cualquier objeto de cualquier lenguaje en un formato estandarizado. Los objetos en Json se representan como un string que utiliza llaves para indicar objetos y corchetes para indicar listas. Básicamente es el mismo formato que hemos usado en JavaScript para objetos y Arrays, pero como strings.

Con el método `JSON.stringify()` podemos convertir cualquier objeto a un string Json:

```
let objeto = { n: 4, a: "hola", c: [1, 2, 3]};
let json = JSON.stringify(objeto);           // json = '{"n":4,"a":"hola","c":[1,2,3]}'
```

Y con el método `parse()` podemos hacer la conversión inversa:

```
objeto = JSON.parse(json);
```

2.5.4. Fechas.

JavaScript trabaja con fechas usando el tipo **Date**. Los objetos de este tipo representan internamente un dato de fecha como los milisegundos transcurridos desde el 01/01/1970, y todos los métodos de fecha usan la zona horaria GMT+0 (UTC), independientemente de que nuestro ordenador muestre una fecha en la zona horaria apropiada.

Las distintas maneras de crear una fecha son las siguientes:

- Para obtener la fecha actual:


```

      hoy = new Date();           // Retorna un objeto Date
      hoy = Date();              // Retorna un string con la fecha actual
      offset = Date.now();       // Retorna un número (los milisegundos desde el 1/1/1970)
      
```
- Para crear una fecha a partir de sus partes (donde enero es el mes 0):


```

      fecha = new Date(año, mes, día, horas, minutos, segundos, milisegundos)
      fecha = new Date(2012, 8, 1); // El 1 de enero del año 2010, con la hora 00:00:00.000
      
```
- Para crear una fecha como un desplazamiento desde el 01/01/1970:


```

      fecha = new Date(milisegundos)
      fecha = new Date(1000);       // El 1/1/1070 pasado 1 segundo
      
```
- Para crear una fecha a partir de un string:


```

      fecha = new Date("September 1, 2012 11:13:00");
      fecha = new Date("2012-09-01T11:13:00");
      
```



```

fecha = new Date("1/9/2012 11:13:00");

fecha = new Date("September 1, 2012");           // formato inglés
fecha = new Date("01/09/2012 05:12:24");          // formato español
fecha = new Date("2012-09-01T05:12:45.012");       // formato UTC
fecha = new Date("2012");                         // primer día del año 2012
fecha = new Date("2012-09");                     // el primer día de septiembre de 2012

```

Una vez creado un objeto `Date` podemos acceder a sus valores:

```

año = fecha.getFullYear();           // retorna el año como un número con 4 dígitos
mes = fecha.getMonth();              // retorna el mes como un número entre 0 (enero) y 11 (diciembre)
día = fecha.getDate();               // retorna el día como un número entre 1 y 31
horas = fecha.getHours();            // retorna un valor entre 0 y 23
minutos = fecha.getMinutes();        // retorna un valor entre 0 y 59
segundos = fecha.getSeconds();       // retorna un valor entre 0 y 59
milisegundos = fecha.getMilliseconds() // retorna un valor entre 0 y 999
offset = fecha.getTime();            // retorna los milisegundos transcurridos desde el 1/1/1970
díaSemana = fecha.getDay();          // retorna el día de la semana como un valor entre 0 y 6

```

También existen métodos `set` para cambiar partes de la fecha: `setFullYear()`, `setHours()`, etc.

Para obtener la representación de una fecha podemos usar los siguientes métodos:

```

strOffset = fecha.toString()         // retorna un string con los milisegundos de desplazamiento
strUTC = fecha.toUTCString()         // retorna la fecha UTC
strISO = fecha.toISOString()         // retorna la fecha en formato ISO
strInglés = fecha.toDateString()     // retorna la fecha en inglés
strLocal = fecha.toLocaleDateString() // retorna la fecha en formato de la región actual
strTime = fecha.toTimeString()       // retorna la parte de la hora
strTime = fecha.toLocaleTimeString() // retorna la parte de la hora para la región actual

```

2.6. Gestión de errores.

Algunas instrucciones de JavaScript pueden provocar errores que hacen que el resto del código de la página deje de ejecutarse.

2.6.1. Captura de errores.

Los errores que se producen en el código JavaScript pueden ser errores de sintaxis cometidos por el programador, errores debidos a datos incorrectos y otras cosas imprevisibles.

Por ejemplo:

```

let x = 7 % 4;
console.log(x);           ← ERROR
console.log("fin");

```

Como `console` no es un objeto existente se producirá un error de tipo `ReferenceError`, y ya no se ejecutará la última instrucción.

Podemos capturar posibles errores con la estructura `try/catch/finally`:

```

try {
  let x = 7 % 4;
  console.log(x);           ← ERROR
  console.log("fin");
} catch(err) {
  console.log = err.message;
}

```

Cuando se provoca un error dentro de un bloque `try`, la ejecución se deriva hacia el bloque `catch`. El bloque `catch` captura el error en una variable de tipo `Error` que nos permite informarnos sobre el error. Los objetos `Error` proporcionan las propiedades `name` (el tipo de error) y `message` (un mensaje descriptivo del error).

A `try/catch` podemos añadir un bloque opcional `finally` cuya instrucciones se ejecutarán siempre, tanto si se produce un error en el bloque `try` como si no.

```

try {
  var x = 7 % 4;
  console.log(x);           ← ERROR
} finally {
  // código que siempre se ejecutará
}

```



```
} catch(err) {  
  console.log = err.message;  
} finally {  
  console.log("fin");  
}
```

O bien podemos utilizar una combinación `try{}finally{}` sin el bloque `catch`. En este caso no se capturarán los errores.

2.6.2. Lanzar errores.

A veces nos puede interesar provocar un error personalizado si se producen determinadas circunstancias que por sí mismas no provocan un error. Por ejemplo, si solicitamos un número y es cero podemos provocar un error con la instrucción `throw`:

```
var valor = prompt("Un número:");  
if (valor == 0) {  
  throw "El valor es cero."  
}
```

Después de `throw` podemos añadir un texto, un número, un booleano o un objeto. Este valor será el capturado por la variable del `catch`.

Después de `throw` podemos usar un objeto de uno de los tipos de errores: `Error`, `EvalError`, `RangeError`, `ReferenceError`, `SyntaxError`, `TypeError` y `URIError`.

